Data Loading (ETL)

Dataset:

Order_ID	Customer_ID	Sales_Amount	Order_Date
O101	C001	4500	12-01-2024
O102	C002	Null	15-01-2024
O103	C003	3200	2024/01/18
O101	C001	4500	12-01-2024
O104	C004	Three Thousand	20-01-2024
O105	C005	5100	25-01-2024

Q1. Data Understanding

All data quality issues present in the dataset that can cause problems during data loading:

Issue 1: Duplicate Primary Key (Order_ID = O101)

- Record 4 (O101, C001, 4500, 12-01-2024) is an exact duplicate of Record 1.
- This violates the PRIMARY KEY constraint — no two rows can have the same Order_ID.

Issue 2: Missing Value in Sales_Amount

- Record 2 (O102) has 'Null' in the Sales_Amount column.
- A NULL in a numeric field causes aggregation errors and may violate NOT NULL constraints.

Issue 3: Invalid Data Type in Sales_Amount

- Record 5 (O104) has 'Three Thousand' — a text string instead of a numeric value.
- This will cause a type-cast failure when loading into a DECIMAL column in SQL.

Issue 4: Inconsistent Date Format in Order_Date

- Records 1, 2, 5, 6 use format: DD-MM-YYYY (e.g., 12-01-2024)
- Record 3 (O103) uses format: YYYY/MM/DD (e.g., 2024/01/18)
- Mixed formats cause date parsing errors and incorrect date comparisons.

Q2. Primary Key Validation

a) Is the dataset violating the Primary Key rule?

YES — the dataset is violating the Primary Key rule. Order_ID O101 appears twice in the dataset. A Primary Key must be unique for every record.

b) Which record(s) cause this violation?

- Record 1: Order_ID = O101, Customer_ID = C001, Sales_Amount = 4500, Order_Date = 12-01-2024
- Record 4: Order_ID = O101, Customer_ID = C001, Sales_Amount = 4500, Order_Date = 12-01-2024 (DUPLICATE)

Both records are identical. If loaded into a SQL table with Order_ID as PRIMARY KEY, the INSERT will fail with a Duplicate entry error.

Q3. Missing Value Analysis

Which column(s) contain missing values?

Column: Sales_Amount — Record O102 has the value 'Null' (missing value).

a) List the affected records:

- Record 2: Order_ID = O102, Customer_ID = C002, Sales_Amount = NULL, Order_Date = 15-01-2024

b) Why is loading records without handling missing values risky?

- **Aggregation Errors:** SUM(Sales_Amount) and AVG(Sales_Amount) will return incorrect results because SQL ignores NULL in calculations — this corrupts KPIs and reports.
- **NOT NULL Constraint Violation:** If the SQL table defines Sales_Amount as NOT NULL, the record will be rejected entirely, causing the load to fail.
- **BI Dashboard Distortion:** NULL values in BI tools either show as blank or get excluded from totals, making revenue charts misleading.
- **Business Logic Failures:** Any logic like 'flag orders above 3000' will silently skip the NULL record, leading to missed insights.

Q4. Data Type Validation

a) Which record(s) will fail numeric validation?

- Record 2 (O102): Sales_Amount = 'Null' — NULL is not a numeric value; fails NOT NULL numeric validation.
- Record 5 (O104): Sales_Amount = 'Three Thousand' — A text string; completely fails DECIMAL type validation.

b) What would happen if loaded into SQL table with Sales_Amount as DECIMAL?

- **Record 5 (O104 - 'Three Thousand'):** SQL will throw an error: Incorrect DECIMAL value. The record will be rejected and the load will fail or be skipped.
- **Record 2 (O102 - NULL):** If the column has NOT NULL constraint, load fails. Without NOT NULL, it stores as NULL but breaks aggregate calculations.
- **Overall impact:** The ETL pipeline may halt entirely or skip bad records, leading to incomplete data in the database.

Q5. Date Format Consistency

a) All date formats present in the dataset:

- Format 1: DD-MM-YYYY — Used by O101, O102, O104, O105 (e.g., 12-01-2024, 15-01-2024)
- Format 2: YYYY/MM/DD — Used by O103 only (e.g., 2024/01/18)

b) Why is this a problem during data loading?

- **Parsing Failure:** SQL DATE columns expect a single consistent format. Mixed formats cause the database parser to reject records or misinterpret them.
- **Silent Misinterpretation:** '12-01-2024' could be read as December 1st instead of January 12th by some parsers — storing completely wrong dates with no error.
- **Sort/Filter Errors:** Mixed date strings prevent proper chronological sorting. Queries like WHERE Order_Date BETWEEN ... return incorrect results.
- **ETL Pipeline Crash:** Most ETL tools will throw a ValueError when trying to parse 2024/01/18 if the expected format is DD-MM-YYYY.

Q6. Load Readiness Decision

a) Should this dataset be loaded directly into the database?

NO — This dataset should NOT be loaded directly.

b) Justification (at least three reasons):

- **Reason 1 — Duplicate Primary Key:** Order_ID O101 appears twice. Loading this will cause a PRIMARY KEY constraint violation error or result in duplicate data, leading to incorrect counts and aggregations.
- **Reason 2 — Invalid Data Type:** 'Three Thousand' in Sales_Amount (O104) cannot be stored in a DECIMAL column. This will cause the SQL INSERT to fail with a type mismatch error.
- **Reason 3 — Missing Values:** O102 has NULL in Sales_Amount. Loading without handling this will corrupt aggregate functions and violate NOT NULL constraints if defined.
- **Reason 4 — Inconsistent Date Formats:** Two different date formats exist in Order_Date. Loading without normalization will cause parsing errors or store incorrect dates, breaking time-based analysis.

Q7. Pre-Load Validation Checklist

The following validation checks must be performed before loading this dataset:

Primary Key Validation

- Check for duplicate values in Order_ID column.
- Ensure every Order_ID is unique and non-NULL.

NULL / Missing Value Check

- Scan all columns for NULL, empty string, or placeholder 'Null' text values.
- Decide: impute with a default value, reject, or quarantine the record.

Data Type Validation

- Verify Sales_Amount contains only numeric values (integers or decimals).
- Reject or flag any text/string values like 'Three Thousand'.

Date Format Validation

- Check all Order_Date values match a single standard format (YYYY-MM-DD recommended).
- Convert all non-standard formats before loading.

Duplicate Record Check

- Check for fully duplicate rows where all columns are identical.

Record Count Validation

- Verify the number of source records matches the expected count after cleaning.

Q8. Cleaning Strategy

Step-by-step cleaning actions to make this dataset load-ready:

Step 1: Remove Duplicate Records

- Delete Record 4 (O101, C001, 4500, 12-01-2024) — exact duplicate of Record 1.
- Keep only the first occurrence of O101.

Step 2: Handle Missing Values

- For Record 2 (O102), Sales_Amount = NULL.
- Option A: Impute with 0 or the column average if business rules allow.
- Option B: Reject the record and log it in an error/quarantine table for manual review.

Step 3: Fix Invalid Data Types

- For Record 5 (O104), convert 'Three Thousand' to its numeric equivalent: 3000.
- If the value cannot be verified, reject the record and log it.

Step 4: Standardize Date Formats

- Convert all Order_Date values to ISO standard format: YYYY-MM-DD.
- 12-01-2024 becomes 2024-01-12
- 2024/01/18 becomes 2024-01-18

Step 5: Validate After Cleaning

- Re-run the full pre-load validation checklist on the cleaned dataset.
- Confirm 0 errors before proceeding to load.

Step 6: Load Clean Data

- Load the 5 valid cleaned records into the database.
- Log summary: total records, loaded records, rejected records.

Q9. Loading Strategy Selection

a) Should a Full Load or Incremental Load be used?

Recommended: Incremental Load (Delta Load)

b) Justification:

- **Daily Sales Data Nature:** Only new or updated records are added each day. Re-loading all historical records every day (Full Load) is unnecessary and wasteful.

- **Performance Efficiency:** Incremental Load processes only new/changed records since the last load, reducing processing time and system load significantly.
- **Avoids Duplication Risk:** Full Load without proper deduplication logic would re-insert all old records daily, causing duplicates. Incremental Load uses a watermark to fetch only new records.
- **Scalability:** As the dataset grows over months/years, Incremental Load remains fast. Full Load would become increasingly slow and resource-intensive.

Implementation: Use Order_Date as the watermark column. Each daily run loads only records where Order_Date = current date or > last loaded date.

Q10. BI Impact Scenario

Assuming the uncleaned dataset was loaded directly and connected to a BI dashboard:

a) What incorrect results might appear in Total Sales KPI?

- Total Sales would be inflated due to the duplicate record O101 — 4500 counted twice instead of once.
- 'Three Thousand' (O104) would either be excluded from SUM causing understated revenue, or cause an error.
- NULL in O102 Sales_Amount would be excluded from SUM by default, making total sales lower than actual.

b) Which records specifically would cause misleading insights?

- O101 (Duplicate): Counted twice — inflates order count and total revenue by 4500.
- O102 (NULL Sales): Excluded from revenue calculations — understates total sales.
- O104 ('Three Thousand'): Cannot be summed — either excluded or causes dashboard errors, understating revenue by 3000.

c) Why would BI tools not detect these issues automatically?

- BI tools like Power BI, Tableau, and Looker are visualization engines — they trust the data they receive and do not perform data quality validation.
- Duplicates: BI tools display all rows from the source. They have no built-in rule to know that O101 should appear only once.
- NULL Handling: BI tools silently skip NULLs in SUM/AVG. No warning is shown — the number just becomes lower.
- Type Errors: If 'Three Thousand' is stored as VARCHAR, BI tools treat it as a category not a measure — it disappears from numeric KPIs silently.
- Root Cause: Data quality is the responsibility of the ETL layer, not the BI layer. Garbage In = Garbage Out.